# PROGRAMACION MULTIMEDIA

TAREA I UD2

ALUMNO CESUR 25/26

Alejandro Muñoz de la Sierra

PROFESOR

Gonzalo Pérez Crespo

# I N T R O D U C C I O N

Este caso práctico trata sobre un tema central y cambiante del desarrollo móvil: el uso y la gestión de contenido multimedia. Usamos el entorno de Android Studio para crear esta aplicación unificada. Combinamos la lógica de programación en Java con el diseño de interfaces en XML. El programa puede reproducir audio desde dos fuentes de datos con orígenes muy diferentes:

Conexión remota por streaming. El sistema usa un archivo guardado en un servidor externo por Internet.

Almacenamiento local nativo. El programa reproduce un archivo guardado en los recursos internos dentro de la carpeta res/raw de la aplicación.

El punto técnico central y el reto real de este trabajo no fue solo emitir sonido por el dispositivo. Queríamos conocer a fondo y dominar la clase MediaPlayer de Android para el futuro. Programamos una lógica capaz de manejar la máquina de estados interna que rige este reproductor de sonido. Controlamos los pasos de creación, preparación, inicio de reproducción, pausa y parada final.

Aplicamos buenos métodos de desarrollo profesional para lograr este objetivo técnico. Declaramos los permisos de red necesarios en el archivo de texto AndroidManifest.xml. Usamos el método asíncrono prepareAsync para no detener el proceso principal de la interfaz de usuario. También creamos un diseño visual claro, fácil de usar y adaptable basado en un LinearLayout ordenado.

# 0 1

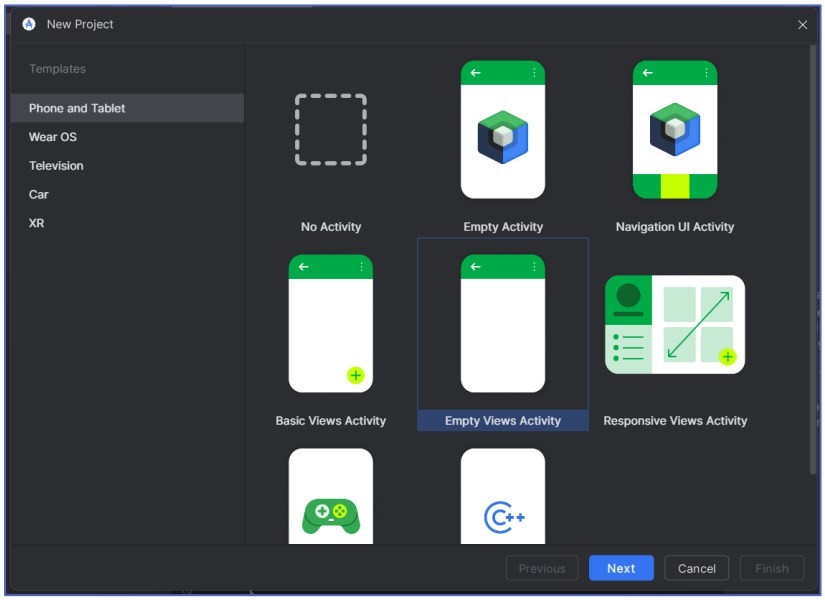
# PREPARACIÓN Y CONFIGURACIÓN DEL ENTORNO DE DESARROLLO

Para garantizar la reproducibilidad de este caso práctico, documentamos a continuación el proceso exacto de configuración del entorno de desarrollo (IDE). Un proyecto en Android no consiste en un único archivo de código, sino que requiere una estructura de directorios y configuraciones a nivel de sistema que deben prepararse antes de programar la lógica multimedia.

## 1. Creación e Inicialización del Proyecto

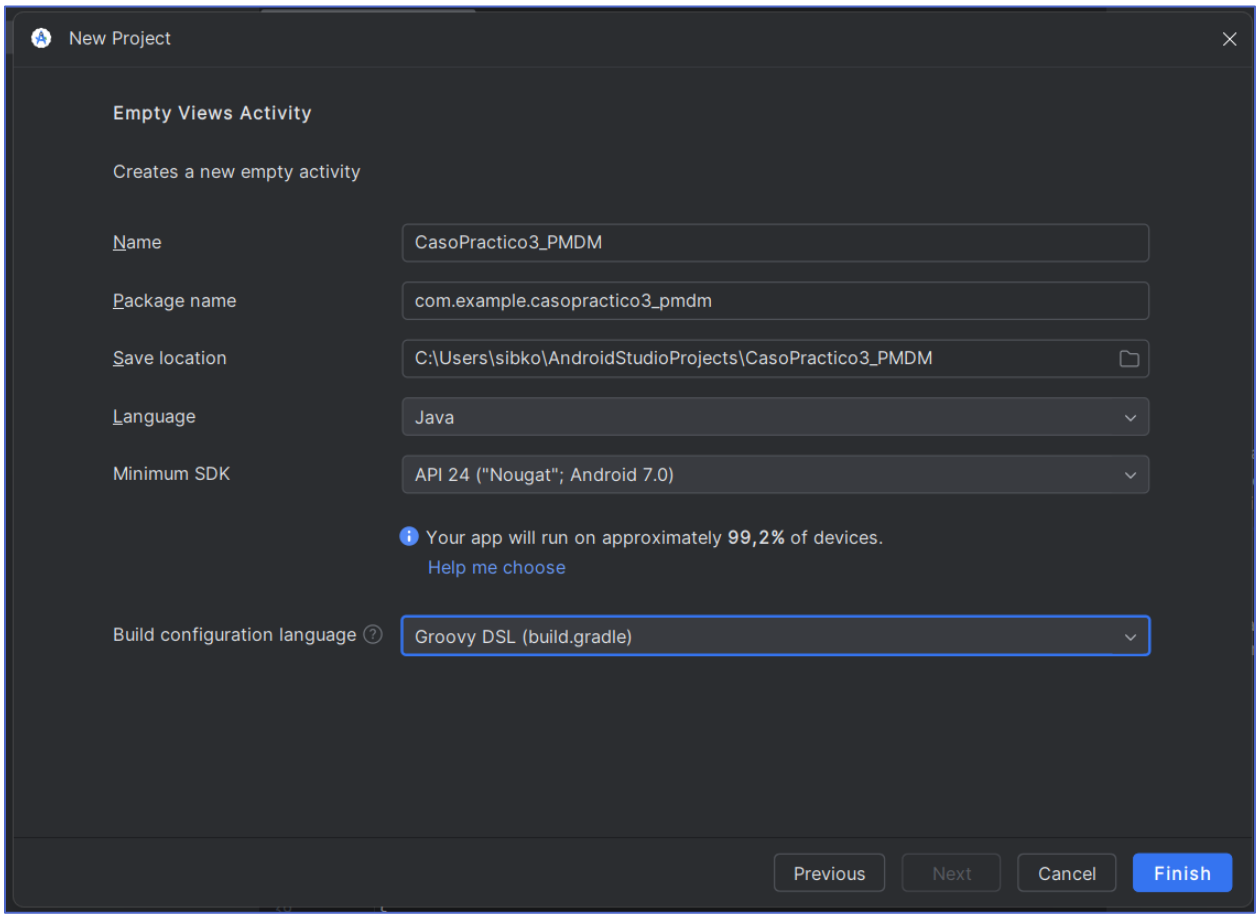
El primer paso es levantar los cimientos de nuestra aplicación en Android Studio asegurándonos de elegir las tecnologías correctas para este módulo:

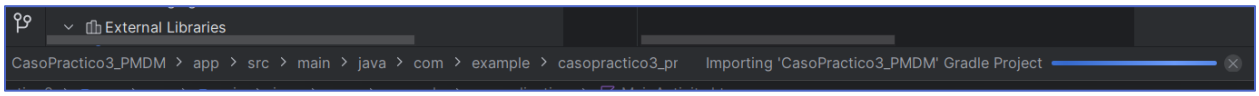
Selección de la Plantilla (El paradigma visual): Al iniciar un nuevo proyecto ("**New Project**"), el IDE nos presenta múltiples arquitecturas base. Es un **error** crítico seleccionar la opción genérica "**Empty Activity**", ya que en las versiones modernas de Android Studio esto fuerza el uso del nuevo paradigma declarativo Jetpack Compose. Para mantener la **compatibilidad** con el temario y el sistema clásico basado en **XML**, seleccionamos estrictamente "**Empty Views Activity**".



**Nomenclatura y Lenguaje:** Nombramos la aplicación (ej. CasoPractico3_PMDM). En el apartado de **lenguaje**, nos aseguramos de cambiar el valor por defecto (Kotlin) para seleccionar **Java**, garantizando la compatibilidad con los ejemplos de la unidad.

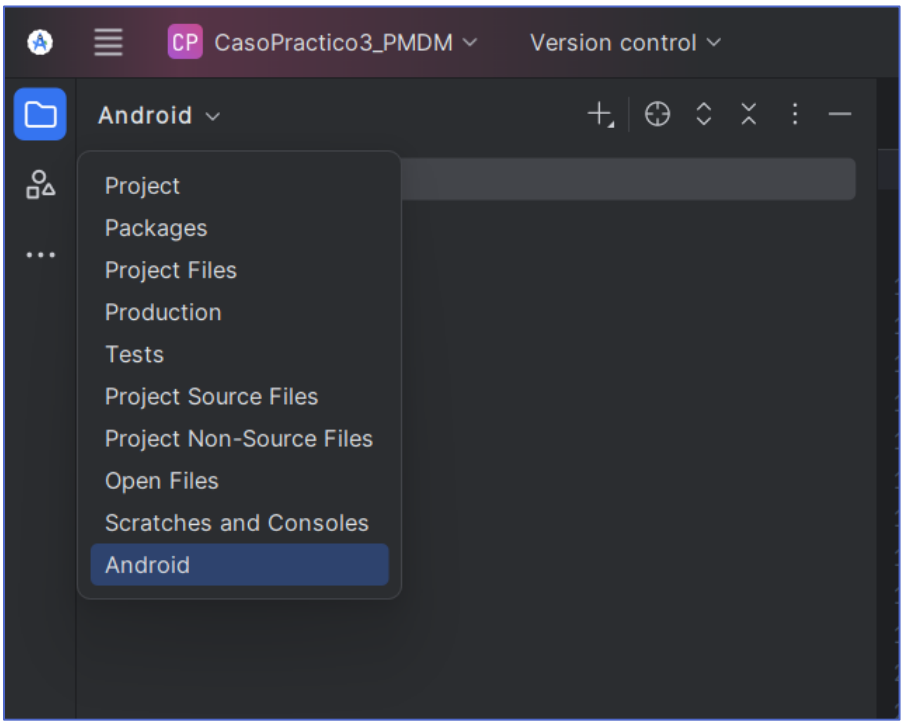
**Minimum SDK:** Mantenemos la **API 24 (o superior)** por defecto, la cual ofrece un equilibrio perfecto entre **compatibilidad** de dispositivos antiguos y **soporte nativo** para las librerías modernas de **android.media**. Al pulsar **Finish**, **Gradle** (el sistema de construcción) descargará las dependencias y estructurará el proyecto.



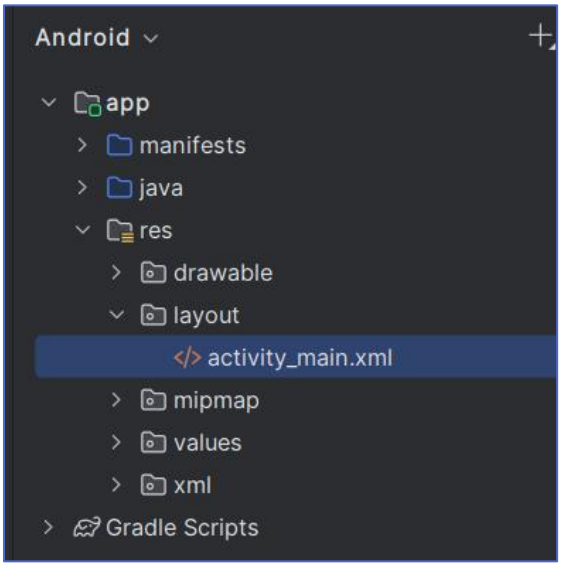
## 2. Arquitectura del Proyecto (Separación de Responsabilidades)

Una vez cargado el proyecto y con la **vista** del explorador ajustada en modo "**Android**", procedemos a trabajar bajo un modelo de separación de responsabilidades, dividiendo nuestro desarrollo en dos "mundos":




**La Capa Visual (El Diseño / Frontend):** Todo lo relacionado con la interfaz de usuario se desarrolla en archivos de marcado .**xml**.

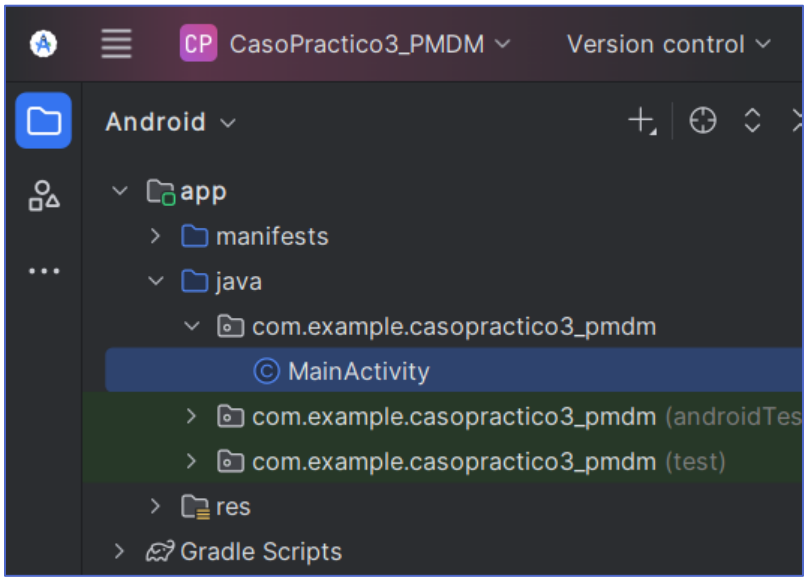
**Ruta:** app > res > layout > **activity_main.xml**.



**Función:** Aquí es donde diseñamos la pantalla, arrastrando nuestros **botones** (**Play, Pause, Stop**) desde la paleta visual **o** escribiendo el código **XML** para definir su tamaño, posición (constraints) e identificadores (@+id/).

**La Capa Lógica (El Cerebro / Backend):** Toda la lógica de control, los eventos de los botones y la invocación al **MediaPlayer** residen en archivos .**java**.

**Ruta:** app > java > com.example.casopractico3_pmdm > **MainActivity.java**.

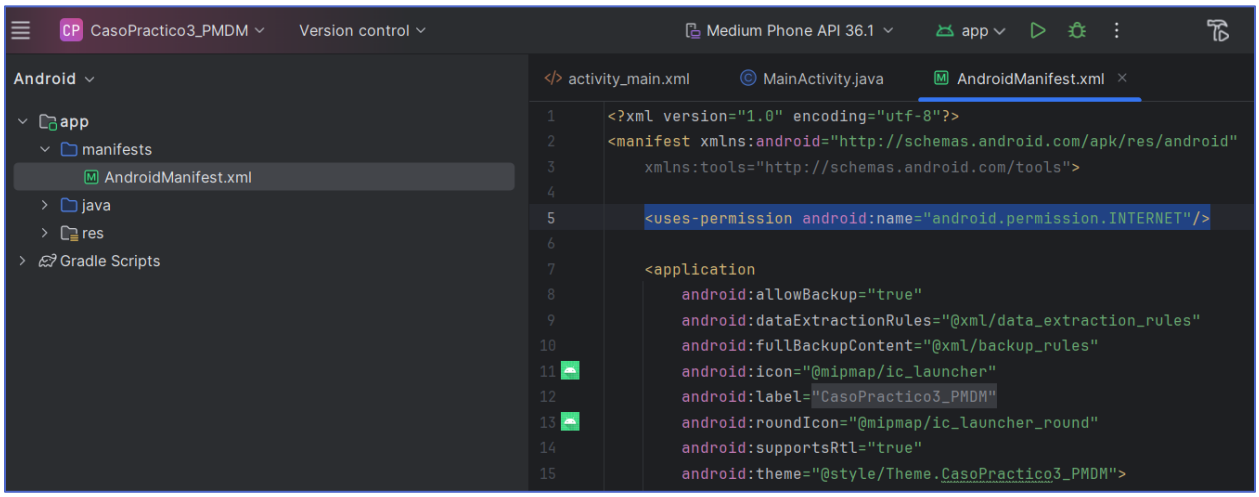



**Función:** Aquí programamos los bloques **try-catch** y el **ciclo de vida** del reproductor. Es el código que dicta cómo debe reaccionar la aplicación cuando el usuario interactúa con el XML.

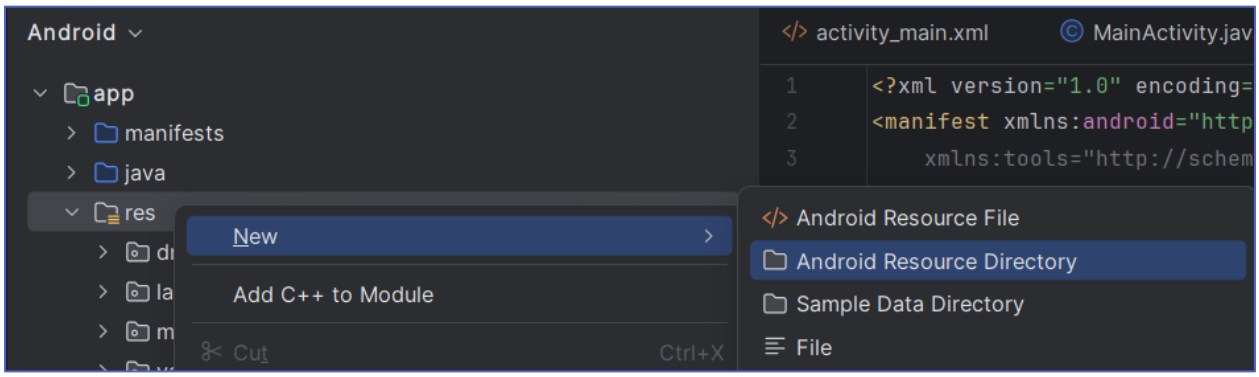
## 3. Configuraciones Críticas del Ejercicio

Para que el **objeto MediaPlayer** pueda acceder a las fuentes de audio (tanto remotas como locales), el sistema operativo Android nos obliga a declarar nuestras intenciones explícitamente:

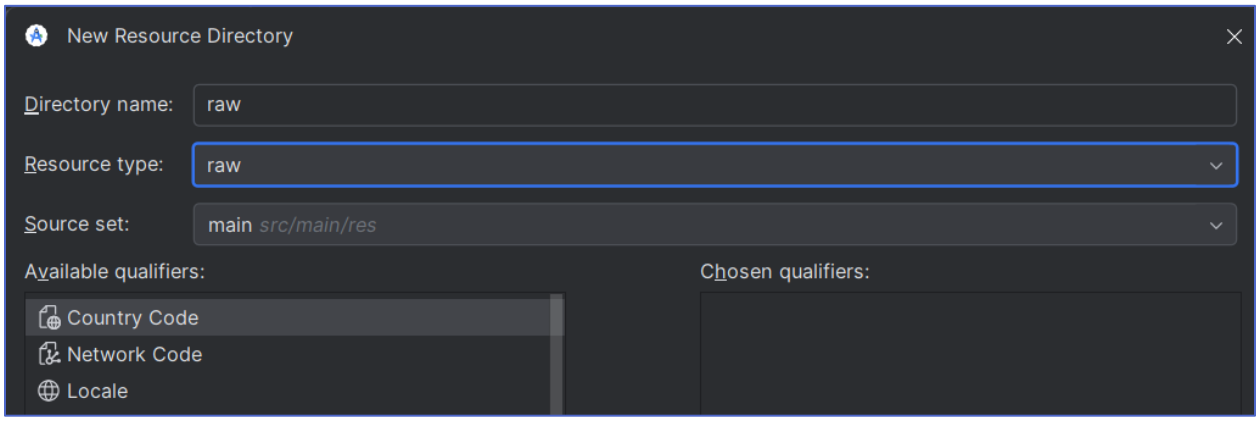
**Declaración de Permisos de Red** (**Ejercicio 1**): Android bloquea por defecto la salida a internet de cualquier app. Para **autorizar la descarga del streaming** de **audio**, navegamos a **app > manifests > AndroidManifest.xml**. Justo debajo de la etiqueta principal <manifest> y antes de <application>, inyectamos la **directiva de seguridad:**
**<uses-permission android:name="android.permission.INTERNET"/>**



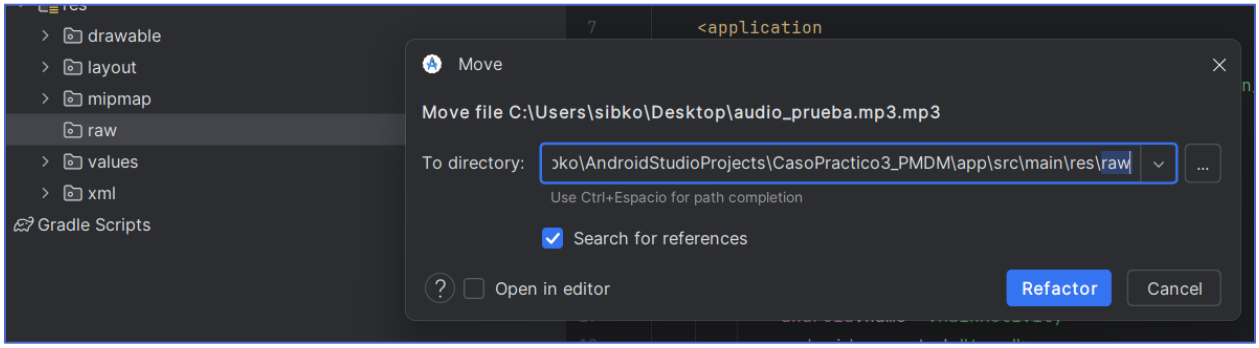

**Directorio de Recursos Nativos (Ejercicio 2):** Para el **audio local**, necesitamos **empaquetar el .mp3** dentro del propio .**apk** final. Para ello, hacemos **clic derecho** sobre la carpeta **res > New > Android Resource Directory**.

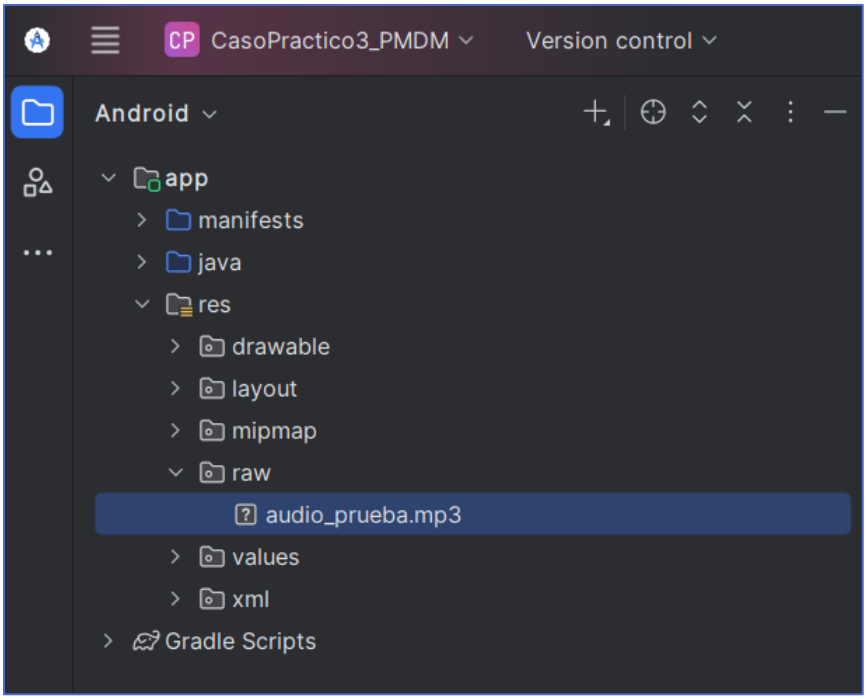


Seleccionamos el tipo **raw** y confirmamos.




Una vez creado el directorio (app/src/main/res/raw/), **pegamos dentro nuestro archivo** de pruebas y lo renombramos a **audio_prueba.mp3** (en Android, los nombres de recursos no pueden contener mayúsculas ni espacios).



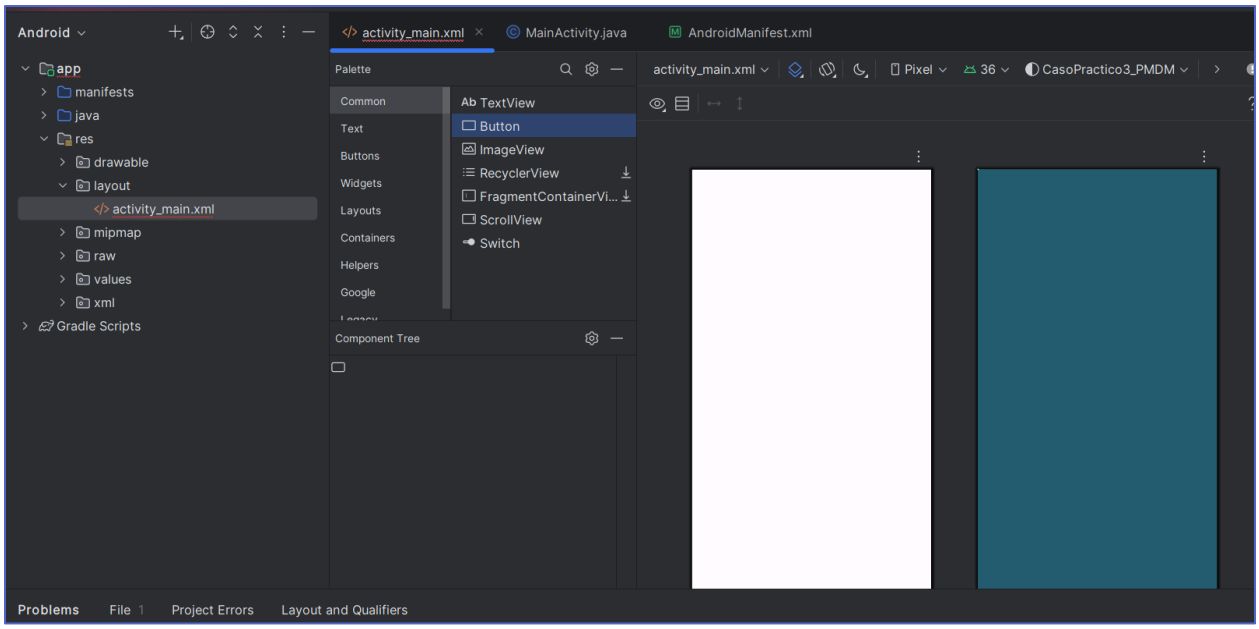
# 0 2

# EJERCICIO 1: REPRODUCCIÓN DE AUDIO DESDE LA WEB

Queremos reproducir un archivo remoto en este escenario. Programamos los **botones PLAY, PAUSA y STOP**. La carga **web** es **asíncrona** y puede fallar. El WiFi falla a veces o el servidor no responde. Un bloque **try-catch** protege el proceso de preparación.

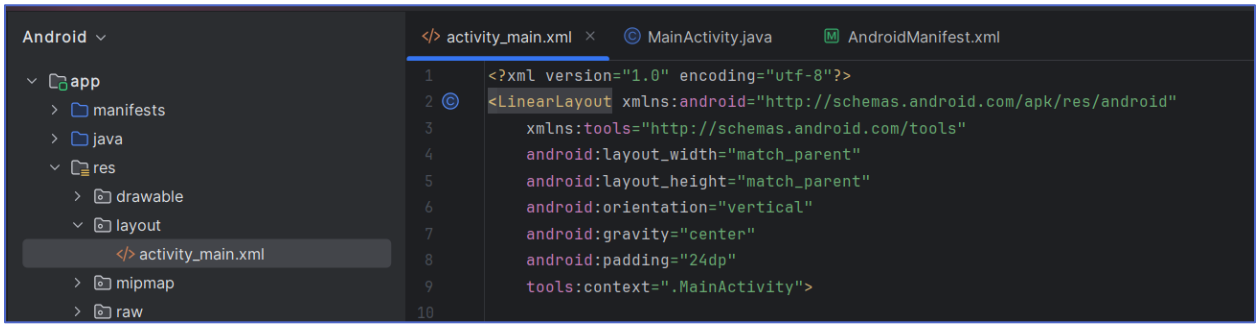
## Diseño de la Interfaz de Usuario (Capa Visual en XML)

Vamos al archivo **activity_main.xml** y nos abre la interfaz gráfica de diseño. Aquí podríamos arrastrar elementos a las pantallas para crear la interfaz.




**No usamos los editores visuales** de arrastrar y soltar. Esas herramientas suelen crear mucho código que no sirve. Nuestro diseño se divide en los elementos clave que explicamos ahora.

Seleccionamos la **vista Code**, para ver el código **xml** y poder construir la interfaz desde aquí con mayor precisión y limpieza.



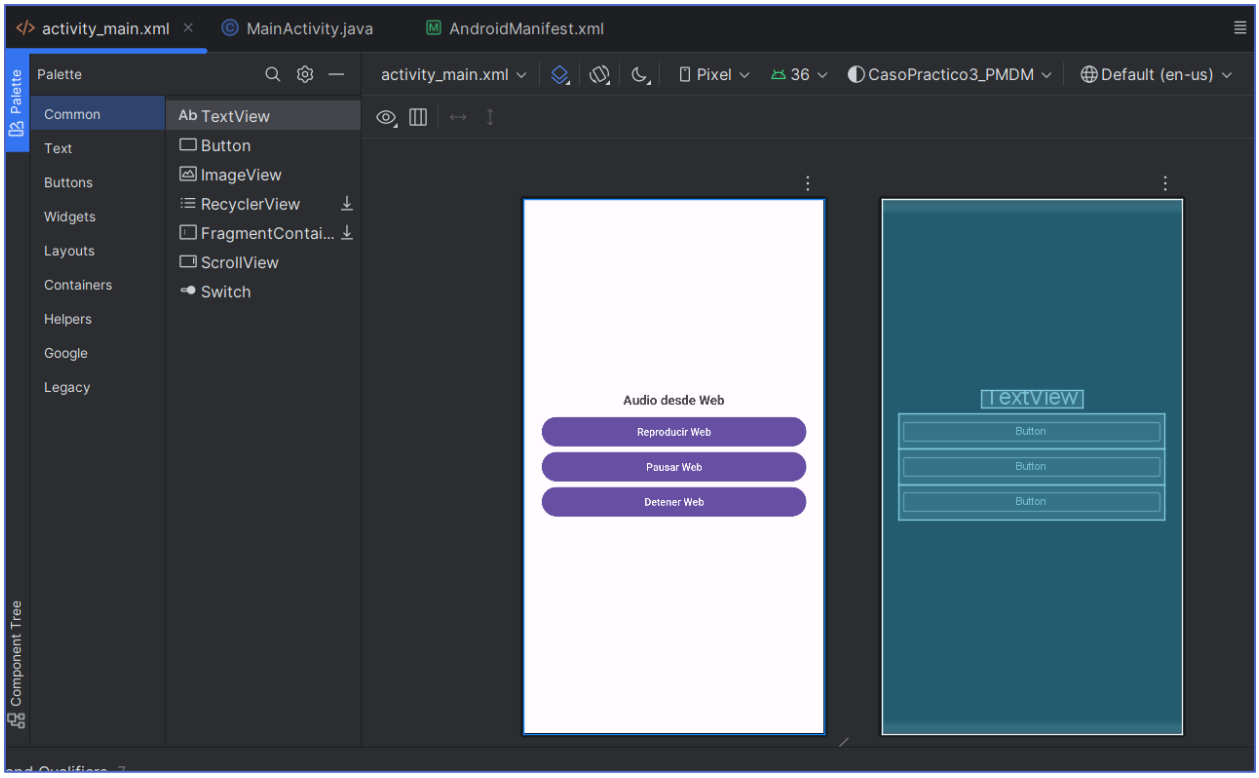

Editamos el código del **archivo activity_main.xml** para construir la capa visual de nuestra aplicación. Escribimos el marcado de forma directa para tener un código limpio.

```
11  <TextView
12      android:layout_width="wrap_content"
13      android:layout_height="wrap_content"
14      android:text="Audio desde Web"
15      android:textSize="18sp"
16      android:textStyle="bold"
17      android:layout_marginBottom="8dp"/>
18
19  <Button
20      android:id="@+id/btnPlayWeb"
21      android:layout_width="match_parent"
22      android:layout_height="wrap_content"
23      android:text="Reproducir Web" />
24
25  <Button
26      android:id="@+id/btnPauseWeb"
27      android:layout_width="match_parent"
28      android:layout_height="wrap_content"
29      android:text="Pausar Web" />
30
31  <Button
32      android:id="@+id/btnStopWeb"
33      android:layout_width="match_parent"
34      android:layout_height="wrap_content"
35      android:text="Detener Web"
36      android:layout_marginBottom="32dp"/>
```

**Contenedor Principal (LinearLayout):** Estructuramos la pantalla con un contenedor invisible. Este objeto ocupa todo el espacio del dispositivo. Definimos su ancho y su alto con el valor match_parent. Los elementos se colocan de arriba a abajo. El atributo android:orientation="vertical" hace que aparezcan así. Centramos el contenido con android:gravity="center". Aplicamos un margen interior de 24dp. Esto evita que los botones toquen los límites de la pantalla del móvil.

**Etiquetas Descriptivas (TextView):** Insertamos textos para que el usuario comprenda el orden de la aplicación. Los textos dicen "Audio desde Web" y "Audio Local". Cambiamos el estilo de estas letras para darles prioridad. Usamos un tamaño de 18sp y el estilo negrita. Estas etiquetas separan las dos secciones del reproductor.

**Controles Interactivos y Binding (Button):** Los botones son la parte central de la interacción. Creamos un botón para cada tarea del reproductor. Asignamos un nombre único a cada uno con android:id. Este paso es la parte más crítica del trabajo. Llamamos al botón del streaming @+id/btnPlayWeb. Este nombre une la vista con el código Java. El programa busca el botón con findViewById(R.id.btnPlayWeb). Después asignamos la acción del clic con un OnClickListener.

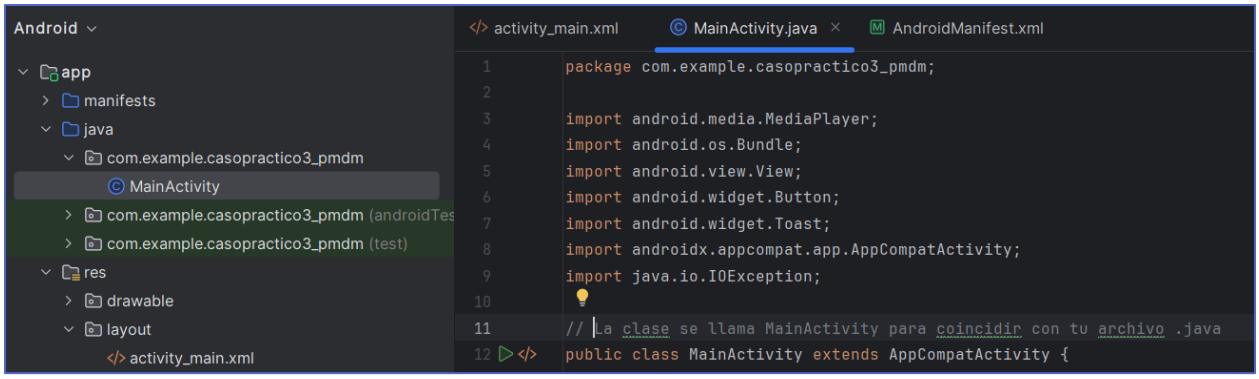



## Lógica de Control y el uso de reset

Definimos la capa visual y ahora damos movimiento a la aplicación. La clase principal en el archivo .java contiene toda la lógica de control del reproductor multimedia. Estructuramos la clase según el ciclo de vida de Android para lograr un código estable y limpio.

### 1. Preparación del Entorno (Paquetes e Importaciones)

Declaramos el paquete al inicio del archivo para definir el espacio de nombres de la app. Después aparece el bloque de importaciones. Java usa tipos estrictos y módulos. El sistema no carga todas las librerías. Por eso el programa funciona rápido por defecto. Avisamos al compilador sobre el uso de clases externas de Android. Usamos controles de interfaz como Button o el motor de audio MediaPlayer.

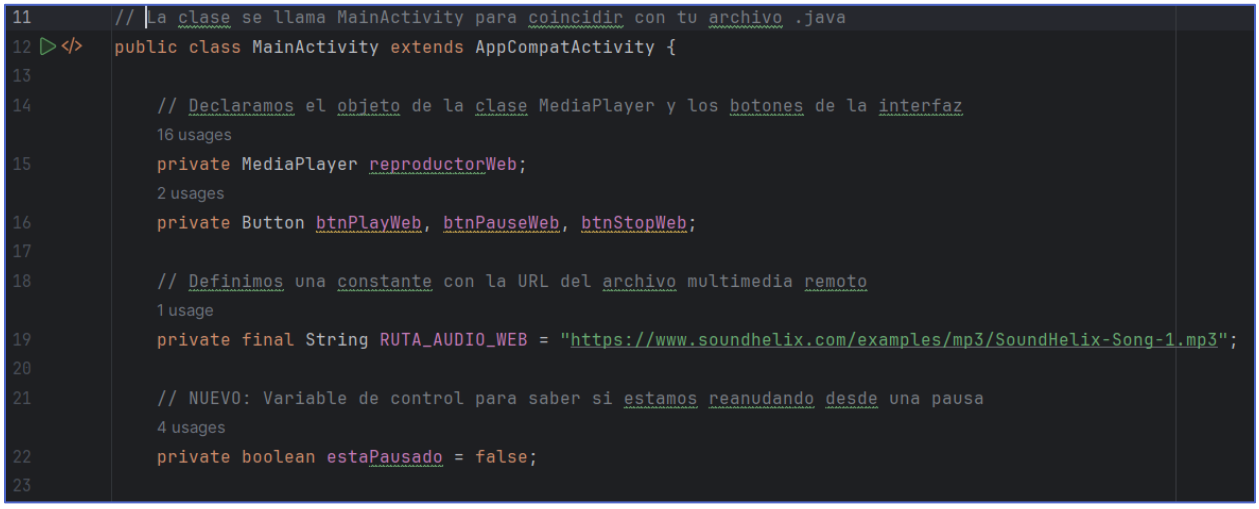



## 2. Ámbito de Variables y Gestión de Estado (Variables Globales)

Declaramos las variables principales debajo de la definición de la clase. Estas incluyen el **objeto reproductorWeb**, los **botones de la interfaz** y la **constante** con la **URL remota**.

Ponemos estas variables a nivel de clase con la palabra **private**. No las creamos dentro de un método concreto o específico. Estas variables funcionan como datos globales para esta actividad. Los **métodos de Play, de Stop o el cierre** de la app **acceden** a ellas fácilmente. El programa mantiene la referencia de los datos en todo momento sin perderla.

Creamos una variable **booleana** llamada **estaPausado** que inicia en **falso**. Esta variable sirve como una marca de estado o máquina de estados simple. La lógica del programa recuerda si el usuario pausó la música. El sistema **distingue** entre una **pausa temporal y un cierre total**.



### 3. Punto de Entrada y Vinculación de Vistas (onCreate)

El método **onCreate() inicia** el **ciclo de vida** de la actividad. El código aquí escrito corre inmediatamente cuando la pantalla aparece.

```java
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        // VINCULACIÓN: Conectamos este Java con nuestro diseño activity_main.xml
        setContentView(R.layout.activity_main);

        // Enlazamos las variables de Java con los IDs que pusimos en el archivo XML
        btnPlayWeb = findViewById(R.id.btnPlayWeb);
        btnPauseWeb = findViewById(R.id.btnPauseWeb);
        btnStopWeb = findViewById(R.id.btnStopWeb);

        // Inicializamos el objeto. El MediaPlayer entra en estado "Idle" (Inactivo)
        reproductorWeb = new MediaPlayer();

```

El método **setContentView() une** la lógica de **Java** con el diseño **XML** que hicimos antes.

Asignamos **nombres únicos** a los **botones** en el archivo **XML** con el código id. Usamos este método para buscar esos elementos físicos en la pantalla. Conectamos los objetos visuales con las variables de Java de forma directa.

Creamos el **objeto** y reservamos **espacio** en la **memoria** para el **reproductor**. El sistema lo mantiene inactivo por ahora. No tiene datos de sonido cargados todavía.

### 4. Lógica de Control y Asincronía (El Botón PLAY)

Ponemos un **detector de pulsaciones** en cada botón. El código revisa dos condiciones de seguridad al presionar **PLAY**.

```java
/* ================================================================
   IMPLEMENTACIÓN DEL BOTÓN PLAY (REPRODUCCIÓN WEB)
   ================================================================ */
btnPlayWeb.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // Condicional de seguridad: Evitamos solapar el audio si ya está sonando
        if (!reproductorWeb.isPlaying()) {

            if (estaPausado) {
                // Si veníamos de darle al Pause, simplemente reanudamos
                reproductorWeb.start();
                estaPausado = false; // Reiniciamos el estado
            } else {
                // Si es la primera vez o venimos de un Stop, cargamos la URL desde cero
                try {
                    // CRÍTICO: Llevamos el objeto al estado "Idle" por si venimos de un STOP
                    reproductorWeb.reset();

                    // Asignamos la ruta remota
                    reproductorWeb.setDataSource(RUTA_AUDIO_WEB);

                    // MEJORA: Preparamos asíncronamente para no bloquear la pantalla
                    // mientras descarga el trozo de canción de internet
                    reproductorWeb.prepareAsync();

```

Revisamos primero que el audio no esté sonando. Esta medida evita errores ante pulsaciones repetidas del usuario. El **sistema no** abrirá **varios sonidos** al mismo tiempo que se pisen entre sí.

Miramos el valor de la variable booleana **estaPausado**.

El comando **start()** funciona con el valor verdadero. El sonido sigue desde el punto exacto de la pausa. Cambiamos la marca de estado a falso después de esto. Preparamos el motor con el valor falso o al venir de un Stop. Usamos **reset()** para borrar la configuración anterior. Ponemos la **dirección de internet** con **setDataSource().** Llamamos a la función **prepareAsync()**.

```java
                    // Le decimos que empiece a sonar justo cuando termine de preparar
                    reproductorWeb.setOnPreparedListener(new MediaPlayer.OnPreparedListener() {
                        @Override
                        public void onPrepared(MediaPlayer mp) {
                            reproductorWeb.start(); // Iniciamos la reproducción
                        }
                    });
                } catch (IOException e) {
                    // Capturamos excepciones si la URL falla o no hay internet
                    e.printStackTrace();
                    Toast.makeText( context: MainActivity.this,  text: "Error de red al cargar el audio", Toast.LENGTH_SHORT).show();
                }
            }
        }
    }
});
```

Usamos **prepareAsync()** por una razón de peso. La carga de audio por internet puede tardar unos segundos. Por eso el método prepare() normal detendría la pantalla del teléfono. Esta **función asíncrona evita** que la interfaz se **congele**. Esto causa un error ANR (Application Not Responding). El modo asíncrono mueve la descarga a un hilo secundario. El sistema usa **un setOnPreparedListener.** Esta función **avisa** que el audio está listo. Entonces el sistema inicia el **método start().**

### 5. Control de interrupciones (Botones PAUSA y PARAR)

La lógica de estos controles se basa en cambiar los estados de forma segura.
**PAUSA:** Revisamos que el reproductor de audio exista y funcione. El sistema cambia el marcador estaPausado = true al pausar el sonido.

```
81
82          /* ================================================================
83             IMPLEMENTACIÓN DEL BOTÓN PAUSE (REPRODUCCIÓN WEB)
84             ================================================================ */
85          btnPauseWeb.setOnClickListener(new View.OnClickListener() {
86              @Override
87              public void onClick(View v) {
88                  // Programación defensiva: Solo pausamos si está reproduciendo.
89                  if (reproductorWeb != null && reproductorWeb.isPlaying()) {
90                      reproductorWeb.pause();
91                      estaPausado = true; // Guardamos el estado para que el Play sepa reanudar
92                  }
93              }
94          });
95
```

**PARAR:** El motor multimedia se detiene por completo en este paso. El punto de reproducción regresa al inicio del archivo. El marcador estaPausado = false se activa. Esto permite que el próximo Play cargue la canción desde el principio.

```
96          /* ================================================================
97             IMPLEMENTACIÓN DEL BOTÓN STOP (REPRODUCCIÓN WEB)
98             ================================================================ */
99          btnStopWeb.setOnClickListener(new View.OnClickListener() {
100             @Override
101             public void onClick(View v) {
102                 // Verificamos que el objeto no sea nulo y esté haciendo algo
103                 if (reproductorWeb != null) {
104                     reproductorWeb.stop();
105                     estaPausado = false; // Al detener, rompemos el estado de pausa
106                 }
107             }
108         });
109     }
110
```

### 6. Evitar fugas de memoria (onDestroy)

El sistema operativo llama al método onDestroy() antes de eliminar la actividad del teléfono. Esto ocurre, por ejemplo, cuando el usuario cierra la aplicación en la pantalla de multitarea.

```
111         // Buena práctica: Liberación de recursos vinculada al ciclo de vida
112         @Override
113         protected void onDestroy() {
114             super.onDestroy();
115             // Si el usuario cierra la aplicación, liberamos la memoria y el hardware de audio
116             if (reproductorWeb != null) {
117                 reproductorWeb.release();
118                 reproductorWeb = null;
119             }
120         }
121     }
```

**Liberar recursos:** El objeto MediaPlayer gasta una gran cantidad de recursos físicos del hardware. El sistema usa decodificadores C++ nativos dentro del teléfono móvil. Una fuga de memoria ocurre si la aplicación se cierra sin destruir este objeto de forma manual. Este fallo reduce el rendimiento del dispositivo móvil. El error gasta la batería en segundo plano. Al llamar a reproductorWeb.release(), el programa devuelve todos los recursos al sistema operativo de forma limpia.

# 03

# EJERCICIO 2: REPRODUCCIÓN DE AUDIO FICHERO LOCAL (RES/RAW)

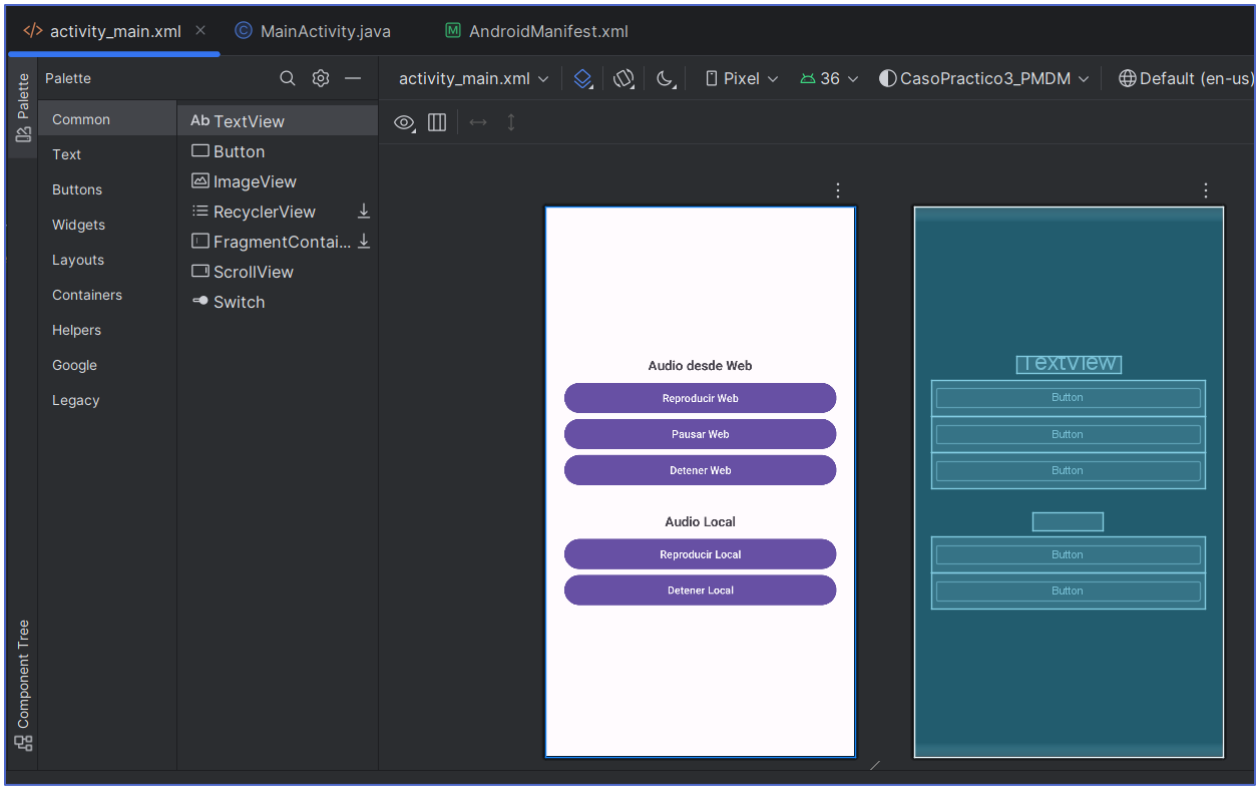
En este caso como ya hemos explicado las estructura general en el ejercicio anterior, vamos a realizar este caso , complementando los archivos con código extra que cumpla con la funcionalidad nueva.

**Diseño de la Interfaz de Usuario (Capa Visual en XML)**

Añadimos la parte del código XML para la interfaz del reproductor local.

**Separación Visual y UX (Márgenes):** Ayudamos al usuario a distinguir las zonas de la pantalla. Separamos la parte de internet de la parte local. Aplicamos un margen exterior de 32dp al último botón de la web. Este espacio en blanco crea un hueco visual claro. La pantalla se ve limpia con esta estructura. El resultado parece profesional y es fácil de usar.

```
38      <TextView
39          android:layout_width="wrap_content"
40          android:layout_height="wrap_content"
41          android:text="Audio Local"
42          android:textSize="18sp"
43          android:textStyle="bold"
44          android:layout_marginBottom="8dp"/>
45
46      <Button
47          android:id="@+id/btnPlayLocal"
48          android:layout_width="match_parent"
49          android:layout_height="wrap_content"
50          android:text="Reproducir Local" />
51
52      <Button
53          android:id="@+id/btnStopLocal"
54          android:layout_width="match_parent"
55          android:layout_height="wrap_content"
56          android:text="Detener Local" />
57
58  </LinearLayout>
```




## Configuración Lógica para Archivos Locales (Recursos raw)

El segundo objetivo de nuestra práctica es lograr la reproducción de un audio guardado en el archivo .apk del programa. El resultado final es igual al del caso web: el sonido debe sonar. Pero la **estructura y** el **manejo** del **estado** del **MediaPlayer** son **distintos** cuando usamos recursos locales en res/raw.

### 1. Declaración de Referencias Locales

El paquete es el mismo al igual que las importaciones. El código empieza a añadirse a partir de la declaración de variables. Definimos las nuevas variables en la parte de arriba de la clase principal. Creamos espacios para los **nuevos** elementos: **reproductorLocal, btnPlayLocal y btnStopLocal.**

```
22
23  // --- Variables para el reproductor Local ---
    12 usages
24  private MediaPlayer reproductorLocal;
25  // Quitamos el btnPauseLocal de aquí porque en nuestro XML decidimos no poner botón de pausa local
    2 usages
26  private Button btnPlayLocal, btnStopLocal;
27
```

**Justificación de Diseño: No pusimos un botón de pausa** en este bloque a propósito. No incluimos ese botón en el diseño visual de activity_main.xml. Al intentar enlazar un botón en el código Java que no está en el XML, el programa fallará. Un error NullPointerException detendría la aplicación. El código debe reflejar la interfaz de forma exacta.

## 2. Enlace de Vistas (View Binding)

**Unimos** las **variables** de la clase **con** los **componentes** físicos de la pantalla dentro del método **onCreate()**. Usamos la orden **findViewById(R.id.btnPlayLocal)** para este fin. Así creamos un puente de comunicación entre la parte gráfica y la lógica que escribimos en Java.

```
28      @Override
29      protected void onCreate(Bundle savedInstanceState) {
30          super.onCreate(savedInstanceState);
31          setContentView(R.layout.activity_main);
```

```
43          // Botones Local (Buscamos los IDs que definimos en el XML)
44          btnPlayLocal = findViewById(R.id.btnPlayLocal);
45          btnStopLocal = findViewById(R.id.btnStopLocal);
46
```

## 3. Instanciación Síncrona y Patrón Factoría (El Botón PLAY)

La **reproducción local** y el streaming difieren en la preparación del motor de sonido. Los archivos están en la memoria interna del equipo. **La espera es casi nula**. **No** usamos procesos **asíncronos** difíciles de manejar.

**Control de Nulidad:** Al recibir el clic, revisamos el estado de la instancia con la **línea if (reproductorLocal == null).** Esta prueba nos dice si el reproductor está borrado o si es la primera vez que arranca.

**El patrón MediaPlayer.create():** Tras validar la prueba como cierta, pedimos al método **MediaPlayer.create(MainActivity.this, R.raw.audio_prueba)** que fabrique el objeto. Este método es una herramienta veloz y eficaz. En un solo paso crea el reproductor y **busca el archivo** en la carpeta raw. Hace el proceso **prepare()** por dentro y de forma síncrona. Con esto, podemos escribir **reproductorLocal.start()** en la línea siguiente. El audio sonará al instante.

```
100         /* ================================================================
101            LÓGICA DEL REPRODUCTOR LOCAL (código nuevo, adaptado)
102            ================================================================ */
103
104         // --- BOTÓN PLAY LOCAL ---
105         btnPlayLocal.setOnClickListener(new View.OnClickListener() {
106             @Override
107             public void onClick(View v) {
108                 // CASO A: Es la primera vez o venimos de un STOP
109                 if (reproductorLocal == null) {
110                     // Carga el audio de la carpeta raw.
111                     // IMPORTANTE: Asegúramos de que R.raw.audio_prueba coincida con el nombre de archivo mp3
112                     reproductorLocal = MediaPlayer.create( context: MainActivity.this, R.raw.audio_prueba);
113                     reproductorLocal.start();
114                 }
115                 // CASO B: El reproductor existe pero no está sonando (en este diseño no tenemos pausa local,
116                 // pero lo dejamos por si alguna vez añadimos el botón).
117                 else if (!reproductorLocal.isPlaying()) {
118                     reproductorLocal.start();
119                 }
120             }
121         });
122
```

## 4. Estrategia de Destrucción de Estado (El Botón STOP)

**No usamos la técnica reset()** para detener el sonido local. Esa fue la opción del caso web. Elegimos un plan de limpieza directa. Hacemos tres tareas al tocar el botón de parar:

**Interrupción:** Usamos **stop()** para detener el sonido de inmediato.

**Liberación de Hardware:** Usamos **release()** para soltar el archivo de audio del motor. Los archivos .mp3 ocupan mucha memoria RAM. Así el sistema operativo recupera esos recursos.

```java
122
123         // --- BOTÓN STOP LOCAL ---
124         btnStopLocal.setOnClickListener(new View.OnClickListener() {
125             @Override
126             public void onClick(View v) {
127                 if (reproductorLocal != null) {
128                     reproductorLocal.stop();
129                     reproductorLocal.release(); // Liberamos memoria
130                     reproductorLocal = null;    // Lo igualamos a null para que el Play sepa que debe volver a crearlo
131                 }
132             }
133         });
134     }
135
```

**Destrucción del Puntero (reproductorLocal = null;):** Escribimos **reproductorLocal = null.** Este paso es una decisión clave para el orden. Ayudamos al sistema de Android a **limpiar la memoria**. Reiniciamos nuestro estado interno. Tras pulsar **PLAY** otra vez, la validación será verdad y el **objeto se creará de nuevo** para una reproducción limpia. limpia.

## 5. Saneamiento del ciclo de vida (onDestroy)

Mantenemos la consistencia con las buenas prácticas de la programación móvil. Repetimos la validación del cierre al final de nuestro archivo. El método **onDestroy()** comprueba el objeto reproductorLocal en la memoria (!= null). El usuario puede forzar el cierre de la aplicación desde el gestor de tareas del sistema. En ese caso, este bloque ejecuta la función **release().** Luego anula el objeto. Esta medida de seguridad es necesaria para evitar procesos de audio zombis o fantasmas activos en segundo plano. Dichos procesos afectan el **rendimiento del terminal**. El audio agota su batería.

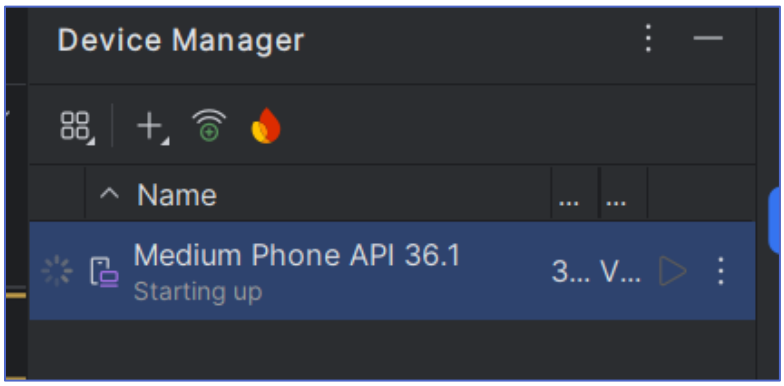
```java
136         /* ================================================================
137            LIMPIEZA AL CERRAR LA APP
138            ================================================================ */
139         @Override
140         protected void onDestroy() {
141             super.onDestroy();
142             // Liberamos el reproductor web
143             if (reproductorWeb != null) {
144                 reproductorWeb.release();
145                 reproductorWeb = null;
146             }
147             // Liberamos también el reproductor local
148             if (reproductorLocal != null) {
149                 reproductorLocal.release();
150                 reproductorLocal = null;
151             }
152         }
153     }
```

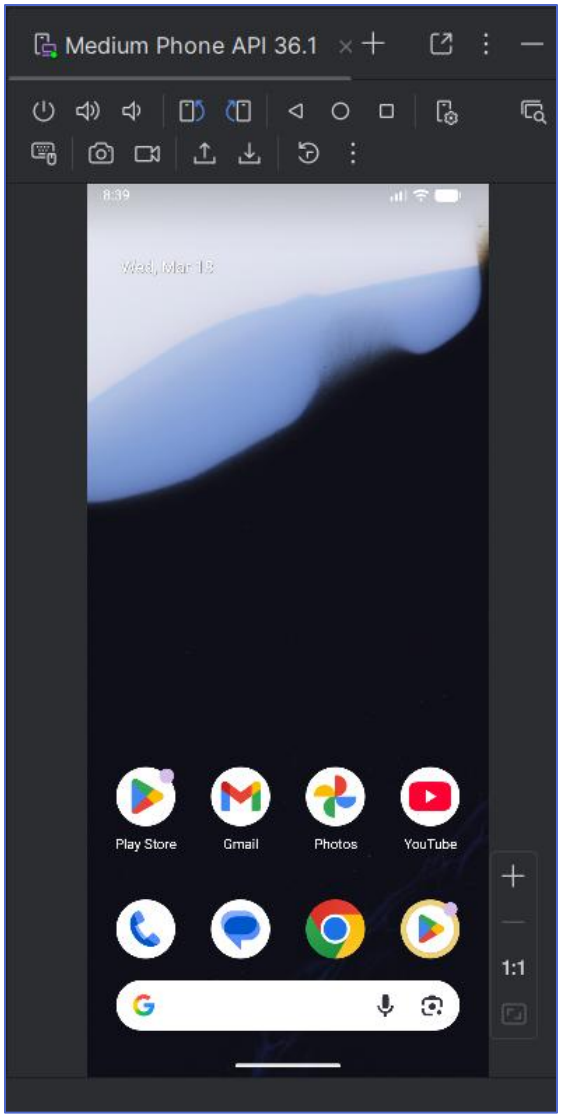
# 04
# COMPILACIÓN Y EJECUCIÓN (EMULADOR AVD)

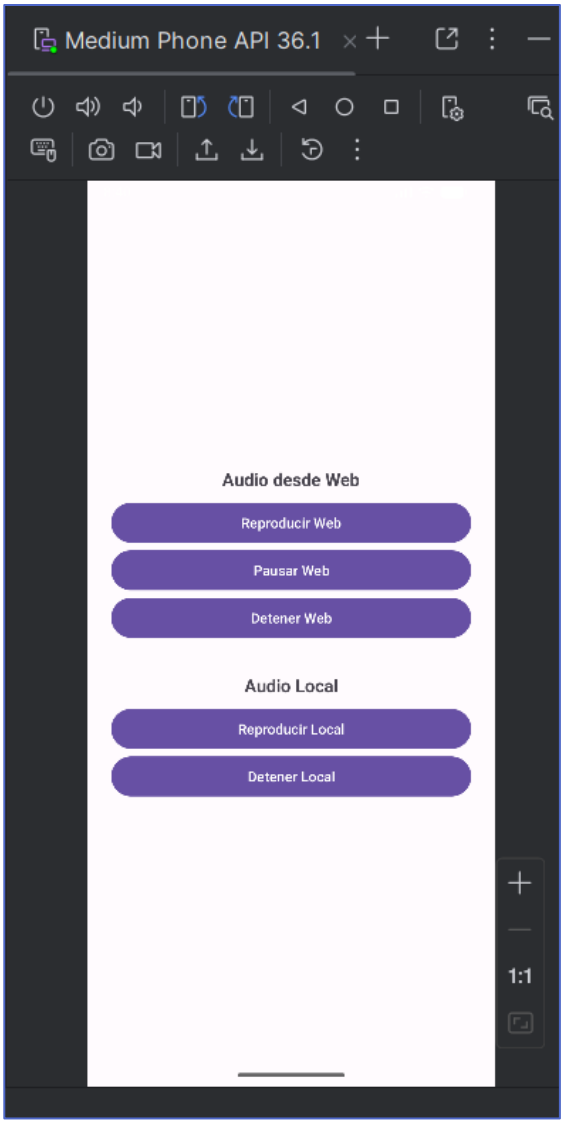
Con el **diseño** (**.xml**), la **lógica** (**.java**) y los **permisos** (**Manifest**) correctamente enlazados, el último paso es la fase de pruebas.

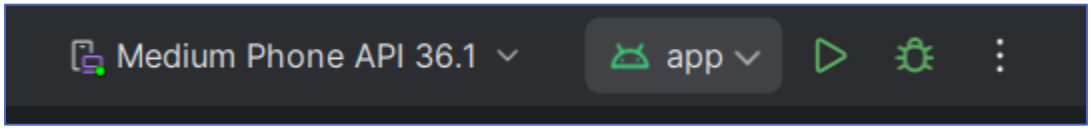
En la barra superior de herramientas de Android Studio, localizamos el menú desplegable de **ejecución** junto al botón verde de **Run ("Play").** Este menú selecciona nuestro dispositivo de pruebas. Utilizaremos el **AVD (Android Virtual Device)** que hemos configurado previamente desde el Device Manager.

Al ejecutar la aplicación, Android Studio iniciará un proceso complejo en segundo plano: **Gradle compilará el código Java a bytecode** (Dalvik/ART), **empaquetará los recursos XML y el MP3** de la carpeta raw en un archivo instalable, y finalmente **abrirá un teléfono virtual** en nuestra pantalla donde desplegará la aplicación interactiva lista para ser testeada.






Vemos que se reproduce el audio perfectamente, tanto en web , como en local. Los métodos para pausa y Stop también funcionan correctamente. Añadimos un enlace a la captura de pantalla en video como prueba de los tests.

Video de la prueba en vivo de la App:

https://drive.google.com/file/d/1iBX6mtXLT99TGeKss972dJMAR36lszwX/view?usp=sharing

# 05
# BUENAS PRÁCTICAS APLICADAS

Usamos normas de calidad para que este código sirva en un entorno de trabajo real. No queremos que sea solo un ejemplo escolar.

**Programación Defensiva:** No usamos los métodos pause() o stop() de forma directa. Primero usamos un if para ver si el reproductor existe y si suena. Esto es clave en Android. Evita que la aplicación se cierre por error. El usuario puede pulsar los botones muy rápido o sin orden y el código lo aguanta. Así evitamos las ventanas de Force Close.

**Gestión de Memoria:** Mantener el hardware de audio activo gasta mucha batería. Esto ocurre si el usuario cierra la aplicación sin liberar los recursos. Usamos el método onDestroy() para ejecutar un release(). Nuestra aplicación cuida los recursos del teléfono y evita fugas de memoria.

**Avisos al Usuario:** Pusimos un Toast en el bloque catch de la parte web. La descarga del audio puede fallar. En ese caso, el sistema avisa al usuario sobre el problema de red. La aplicación no se queda en silencio ni se congela.

# 0 6
# C O N C L U S I O N E S

Logramos crear un reproductor dual que es funcional y operativo. Este proyecto marcó un punto de cambio en nuestra forma de entender la estructura interna de Android. Aprendimos a manejar los recursos del hardware de forma correcta.

Superamos desafíos técnicos de alto nivel. Sincronizamos los actos del usuario que no podemos predecir con la máquina de estados de la clase MediaPlayer. Aprendimos que el reproductor no puede pausarse, reiniciarse o detenerse sin revisar antes su estado actual con el código. Esta fue la clave para usar programación defensiva. Este método evita errores de ejecución como IllegalStateException y el cierre repentino de la aplicación.

El proyecto mostró la gran relevancia de manejar la memoria en los dispositivos móviles. Estudiamos el ciclo de vida de la Activity. Conectamos el método release para borrar objetos multimedia pesados al evento onDestroy de forma estratégica. Así conseguimos que la aplicación no tenga fugas de memoria. También evitamos el gasto de batería por procesos ocultos en segundo plano.

Esta práctica construye bases muy fuertes para nuestro futuro. Unimos el diseño visual en XML con la lógica de control interna en Java. Manejamos el hardware del teléfono con éxito mediante una unión suave. Este conocimiento nos sirve para pasar a proyectos multimedia y aplicaciones interactivas más difíciles en el futuro.

# 0 7
# R E F E R E N C I A S


https://developer.android.com/media/platform/mediaplayer?hl=es-419

https://developer.android.com/reference/android/media/MediaPlayer

https://developer.android.com/develop/ui/views/layout/linear?hl=es-419

https://developer.android.com/guide/components/activities/activity-lifecycle?hl=es

https://developer.android.com/guide/topics/permissions/overview?hl=es

https://m3.material.io/

https://stackoverflow.com/questions/tagged/android-mediaplayer

**Android Studio desde Cero | Crear PlayList con MediaPlayer en Android**
https://www.youtube.com/watch?v=FMpGMKCIOVA

**CURSO: ANDROID desde CERO con KOTLIN - TUTORIAL ANDROID STUDIO [2023]**
https://www.youtube.com/watch?v=vJapzH_46a8